

# Rapport de Projet

Systèmes Informatiques (I-CYSI-002)

Martin Coghetto

Alexandre Daoust

Louis Garlement

18 décembre 2025

# Table des matières

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                      | <b>3</b>  |
| <b>2</b> | <b>Implémentation</b>                                                    | <b>3</b>  |
| 2.1      | Structures de données . . . . .                                          | 3         |
| 2.1.1    | Chaîne de caractères dynamique ( <code>m_string</code> ) . . . . .       | 3         |
| 2.1.2    | Affichage d'une chaîne de caractères . . . . .                           | 4         |
| 2.1.3    | Liste chaînée de chaînes de caractères ( <code>m_list</code> ) . . . . . | 5         |
| 2.2      | Gestion de la LED . . . . .                                              | 6         |
| 2.2.1    | Branchement de la LED . . . . .                                          | 6         |
| 2.2.2    | Implémentation de la LED . . . . .                                       | 7         |
| 2.3      | Architecture et fonctionnement du programme . . . . .                    | 8         |
| 2.3.1    | Point d'entrée ( <code>main.c</code> ) . . . . .                         | 8         |
| 2.3.2    | Contrôle de la LED en utilisant <code>fork(2)</code> . . . . .           | 9         |
| 2.3.3    | Contrôle de la LED en utilisant <code>pthread(3)</code> . . . . .        | 11        |
| 2.3.4    | Contrôle de la LED dans la fonction <code>solve</code> . . . . .         | 14        |
| 2.3.5    | Calcul de la suite de Conway ( <code>math_suite.c</code> ) . . . . .     | 14        |
| 2.4      | Compilation . . . . .                                                    | 18        |
| 2.4.1    | Instructions de compilation . . . . .                                    | 18        |
| <b>3</b> | <b>Validation et tests</b>                                               | <b>19</b> |
| 3.1      | Validation des arguments . . . . .                                       | 19        |
| 3.2      | Validation des résultats . . . . .                                       | 20        |
| 3.3      | Vérification des fuites de mémoires . . . . .                            | 20        |
| <b>4</b> | <b>Performance</b>                                                       | <b>20</b> |
| 4.1      | Méthodologie . . . . .                                                   | 20        |
| 4.2      | Mesure selon l'implémentation . . . . .                                  | 21        |
| 4.3      | Comparaison avec l'implémentation Python . . . . .                       | 22        |
| <b>5</b> | <b>Conclusion</b>                                                        | <b>23</b> |

# 1 Introduction

En addition à l'énoncé initial, ce projet traite la résolution du problème de la suite de Conway de deux autres façons, la première utilisant l'appel système `fork(2)`, et la seconde implémentant le calcul avec la fonction `pthread(3)`.

Ce rapport détaille la réalisation du projet. Il est composé de quatre parties : Implémentation, Validation et tests, Performance et Conclusion.

Dans le respect de la Charte d'utilisation des systèmes d'intelligence artificielle générative de l'UMONS, nous précisons qu'un outil d'intelligence artificielle a été utilisé exclusivement en tant qu'assistant linguistique (relecture et correction syntaxique) sur le rapport écrit. L'intégralité du code, de l'analyse et de la structure du projet est le fruit de notre propre travail.

## 2 Implémentation

L'implémentation du programme a été réalisée directement sur le Raspberry Pi, en utilisant une connexion ssh. Nous avons utilisé `git` afin de versionner le code et sauvegarder une copie du dépôt sur `GitHub`. Pour accéder au Raspberry Pi depuis l'extérieur, celui-ci était connecté à notre réseau Tailscale, permettant à tout les membres du groupe de travailler simultanément sur le Raspberry Pi à distance.

### 2.1 Structures de données

Lors de la conception de notre programme, nous avons tenu compte des limites matérielles du Raspberry Pi. En effet, la suite de Conway explose en taille de manière exponentielle. Avec comme itéré initial 1, 10 itérations de la suite donnent 26 caractères en sortie. Si nous réalisons 70 itérations, la sortie est longue de 234 241 787 caractères. En considérant 1 octet par caractère, cela donne 234 méga-octets rien que pour stocker ou afficher le résultat de la suite. Pour 73 itérations, on retrouve 518 891 359 caractères (519 méga-octets).

L'implémentation de notre programme utilise une structure de données optimisée permettant de gérer ces gros volumes de mémoire nommée `m_string`<sup>1</sup>. Malheureusement, la complexité de l'évolution de la suite de Conway rend impossible la réalisation d'optimisations sur le calcul. Étant donné que pour un itéré  $i$  donné, la taille de l'itéré  $i + 1$  est nécessairement supérieure, cela rend impossible la réutilisation d'une seule allocation de chaîne de caractères ; nous sommes donc forcés d'en allouer deux : une pour l'itéré  $i$  en entrée et une pour la génération de l'itéré  $i + 1$  en sortie.

Afin de gérer le cas `-only-longest` du programme, c'est-à-dire n'afficher que le(s) résultat(s) les plus longs et ayant le plus de chiffres différents, nous avons implémenté une liste chaînée. En effet, étant donné qu'il est possible qu'il y ait plus d'un résultat "le plus long", nous ne pouvions pas simplement stocker le résultat dans une variable ; il nous fallait une manière robuste de gérer une liste de plusieurs résultats. L'implémentation d'une liste doublement chaînée ayant été vue au TP, nous avons décidé d'implémenter une liste chaînée dans notre programme.

#### 2.1.1 Chaîne de caractères dynamique (`m_string`)

Nous avons donc implémenté une chaîne de caractères à allocation dynamique. La structure C présentée ci-dessous contient deux attributs : un pointeur vers le début de la chaîne de caractères nommé `ptr` ainsi que la taille maximale de la chaîne de caractères `max_size`. Le type `size_t` permet de stocker une quantité d'octets comme défini dans `size_t(3type)` ; sur notre Raspberry Pi, celui-ci fait 64 bits, soit 8 octets.

---

1 `typedef struct m_string {`

1. Le préfixe `m` a été choisi de manière totalement arbitraire. Celui-ci représente le "m" de Math ou Mathsuite, et permet d'éviter les potentielles confusions avec des types de données d'autres langages ou bibliothèques lors de la lecture du code.

```
2     char    *ptr;
3     size_t  max_size;
4 }      m_string;
```

---

Nous utilisons `calloc(3)` pour allouer la chaîne de caractères. Par défaut, la chaîne de caractères est initialisée avec une taille d'allocation `max_size` égale à 32 caractères. La taille effective de la chaîne de caractères n'est pas connue à l'avance car la chaîne est délimitée par le caractère `\0`. En utilisant `calloc`, nous nous assurons que le bloc de mémoire pointé par `*ptr` et de taille `max_string` ne contient que le caractère `\0`. La chaîne de caractères est donc de taille nulle.

## Réallocation de la chaîne de caractères

Lorsque la taille allouée de la chaîne ne suffit plus, le programme utilise la fonction `realloc_string`. Celle-ci récupère un pointeur vers la structure `string` à agrandir. Par défaut, nous doublons `max_size` à chaque appel; cela permet d'amortir le coût du redimensionnement `realloc(3)` du bloc de mémoire pour les grandes chaînes. Pour les petites chaînes, cela maintient le compromis du nombre de réallocations assez faible.

Lorsque nous utilisons `realloc(3)`, il est possible que la chaîne de caractères soit déplacée par l'allocateur de mémoire. Supposons que l'allocateur alloue deux chaînes de 32 caractères à la suite. Si l'on veut doubler la taille de la première, il faut la déplacer à la fin de la deuxième, et donc son adresse change. De plus, la fonction `realloc_string` gère les erreurs d'allocations en retournant `NULL (0)` si `realloc` a échoué, ce qui peut se produire s'il n'y a plus de mémoire disponible sur le Raspberry Pi.

---

```
1 char* realloc_string(m_string *string) {
2     char* ptr;
3
4     ptr = realloc(string->ptr, string->max_size*2 * sizeof(char));
5
6     if (ptr == NULL)
7         return NULL;
8
9     string->ptr = ptr;
10
11     memset(string->ptr + string->max_size, 0, string->max_size);
12     string->max_size *= 2;
13
14     return string->ptr + (string->max_size/2);
15 }
```

---

Nous avons également implémenté plusieurs fonctions utilitaires permettant la manipulation de nos chaînes de caractères : `free_string(m_string* s)`, `zero_string(m_string* s)`, `copy_string(m_string* src, m_string* dest)`, `swap_string(m_string* s1, m_string* s2)`, `string_length(m_string *s)`.

### 2.1.2 Affichage d'une chaîne de caractères

La fonction `print_string(FILE* out, m_string* s, size_t n)` permet d'écrire la chaîne de caractères dans le descripteur de fichier associé à "out". Cette dernière prend en paramètre la chaîne de caractères ainsi que sa taille.

Celle-ci remplace l'appel à `fprintf(out, "%s", s.ptr)` fourni par la bibliothèque standard en utilisant directement l'appel système `write(2)`.

En effet, elle permet trois optimisations importantes :

1. Cela ne demande pas de réaliser l'analyse de la chaîne de format passée en argument. Celle-ci contient "%s", demandant l'affichage du premier paramètre en tant que string, ce qui a un coût.

2. Contrairement à `fprintf`, notre fonction `print_string` connaît la taille de la chaîne de caractères à l'avance, et ne doit pas itérer sur chaque caractère jusqu'à trouver un `\0` (`strlen` implicite).
3. Nous réalisons directement un appel système, ce qui évite de passer par le buffer interne de la `libc`; nous écrivons directement dans le fichier.

Cette fonction possède deux garde-fous : le premier vérifie si le descripteur de fichier passé en argument est valide en récupérant son numéro, le second vérifie que la taille de la chaîne de caractères ne dépasse pas la taille allouée.

Étant donné que nous contournons le buffer interne de la `libc`, nous devons nous assurer d'appeler `fflush` avant chaque utilisation de `write(2)`. En effet, pour minimiser le nombre d'appels système sur des petites chaînes de caractères, la `libc` stocke tout dans un tampon interne et ne l'affiche qu'à partir d'un certain seuil. `fflush` garantit que le contenu du buffer interne est écrit dans le fichier demandé avant notre écriture manuelle.

Cette optimisation permet notamment un gain de temps pour les chaînes de caractères de plusieurs centaines de Mo. Rien que récupérer la taille de la chaîne de caractères peut prendre un certain temps.

De plus, cette fonction essaye d'inclure le caractère de saut de ligne directement dans la chaîne de caractères si cela est possible. Dans l'autre cas, elle l'affiche séparément via un appel à `fprintf`; statistiquement cette partie du code ne sera appelée que très rarement. En effet, il faut que la taille de la chaîne de caractères soit exactement égale à sa taille allouée, soit un multiple de 2. Cela crée un effet de bord : il est possible que la chaîne de caractères soit modifiée en sortie de la fonction. Cela ne pose pas de problème dans notre programme, car nous n'affichons qu'une seule fois chaque chaîne.

---

```

1 void print_string(FILE* out, m_string* s, size_t n) {
2     int fd;
3
4     if ((fd = fileno(out)) == -1)
5         return;
6
7     if (n > s->max_size)
8         n = s->max_size;
9
10    // Il reste de la place pour le '\n'
11    // Le rajouter
12    if (n < s->max_size) {
13        s->ptr[n++] = '\n';
14    }
15
16    // Appel système write(2)
17    // Plus rapide que fwrite qui utilise un buffer interne
18    // Et fprintf qui parse la chaîne de caractère pour chercher des %
19    write(fd, s->ptr, n);
20
21    // Si il n'y avait plus de place pour le '\n'
22    if (n >= s->max_size) {
23        fprintf(out, "\n");
24    } // else: on peut retirer le '\n' mais ce n'est pas nécessaire.
25 }
```

---

### 2.1.3 Liste chaînée de chaînes de caractères (`m_list`)

Nous avons implémenté une liste chaînée contenant nos chaînes de caractères `m_string`. Celle-ci est composée d'une structure `m_list_node` représentant un élément de la liste et d'une structure `m_list` servant de tête de liste.

Nous ne stockons pas de références (pointeurs) vers une structure de chaîne de caractères, mais directement une copie de la structure `m_string` dans chaque nœud de la liste. Cela ne pose aucun problème car la structure `m_string` ne contient pas la chaîne de caractères complète, mais l'adresse de début de

la chaîne et la taille totale du bloc alloué. Sur un système 64 bits, la structure a une taille d'exactement 16 octets. Cela signifie que chaque élément de la liste (`sizeof(m_list_node)`) ne fait que 24 octets car il rajoute le stockage d'un pointeur vers l'élément suivant de la liste.

De plus, la liste chaînée n'est utilisée que pour le stockage des meilleurs résultats ; cela signifie donc que chaque chaîne de caractères est considérée comme immuable, donc réaliser une copie de la structure ne pose aucun problème de cohérence.

---

```

1  typedef struct      m_list_node {
2      struct m_list_node* next;
3      m_string      value;
4  }                  m_list_node;
5
6  typedef struct      m_list {
7      m_list_node* head;
8  }                  m_list;

```

---

Nous avons défini quatre fonctions sur la liste : `init_list(m_list* l)`, `push_back(m_list* l, m_string value)`, `clear_list(m_list* l)` et `print_list(FILE* out, m_list* l)`.

La fonction `init_list` permet d'initialiser une structure existante de liste ; elle ne réalise pas d'allocation mémoire, la responsabilité de l'allocation de la structure `m_list` revient à l'appelant.

Afin d'éviter les fuites de mémoire, nous considérons qu'une fois qu'une chaîne de caractères est ajoutée à la fin d'une liste (via `push_back`), la désallocation de la chaîne de caractères (le pointeur interne) ne doit plus être gérée par l'appelant, mais devient une responsabilité de la liste. La fonction `push_back` utilise `malloc(3)` pour allouer un nouvel élément de la liste.

Le seul moyen de supprimer un élément de la liste est d'appeler `clear_list` ; cette fonction désalloue tous les éléments de la liste et appelle la fonction `free_string` sur chaque chaîne de caractères stockée. Nous n'avons pas implémenté de fonction permettant de retirer un élément en particulier de la liste en raison du comportement du programme : nous accumulons tous les éléments étant les "plus grands" (`-only-longest`). Lorsqu'un nouvel élément encore plus grand est découvert, nous devons supprimer tous les éléments de la liste ; ceux-ci sont devenus invalides car nous avons trouvé mieux.

---

```

1  void clear_list(m_list* l) {
2      m_list_node* curr;
3      m_list_node* next;
4
5      curr = l->head;
6
7      while(curr != NULL) {
8          next = curr->next;
9          free_string(&curr->value);
10         free(curr);
11         curr = next;
12     }
13
14     l->head = NULL;
15 }

```

---

## 2.2 Gestion de la LED

### 2.2.1 Branchement de la LED

Contrairement à ce qui est proposé dans l'énoncé du projet, nous avons décidé de brancher la LED sur les pins Pulse Width Modulation (PWM) du Raspberry Pi (voir la table 1). En effet, nous voulions initialement utiliser la fonctionnalité PWM du Raspberry Pi pour pouvoir utiliser n'importe quelle

couleur RGB sur la LED. Or, le Raspberry Pi ne contient que deux canaux PWM différents, ce qui implique que nous ne pouvions pas correctement gérer la LED de cette façon. Nous avons aussi considéré l'utilisation d'un Software PWM, mais celui-ci requiert l'utilisation d'un thread (de la bibliothèque pthread) par canal de couleur. Au final, nous gérons la LED plus simplement en OUTPUT, c'est-à-dire HIGH ou LOW pour chaque canal de couleur.

|              |         |
|--------------|---------|
| Rouge        | GPIO 12 |
| Vert         | GPIO 19 |
| Bleu         | GPIO 13 |
| Alimentation | GPIO 18 |

**TABLE 1** – Pins de la LED

### 2.2.2 Implémentation de la LED

Nous avons considéré plusieurs options pour la gestion de la LED lors de l'exécution du programme, notamment pour la faire clignoter en deux couleurs. Pour chacune des méthodes, nous avons mesuré la performance du programme afin de déterminer laquelle était la plus adéquate à notre projet (voir section Performance).

De manière générale, nous avons créé une structure `Led`, nous permettant de sauvegarder dans une variable les pins de la LED ainsi que de savoir si celle-ci a été initialisée correctement.

---

```

1 typedef struct Led {
2     uint8_t red_pin;
3     uint8_t blue_pin;
4     uint8_t green_pin;
5     uint8_t power_pin;
6 } Led;

```

---

Nous avons aussi créé une autre structure `Color` pour nous permettre de représenter les couleurs usuelles (celles utilisées dans le programme) par des macros.

---

```

1 typedef struct Color {
2     uint8_t red;
3     uint8_t green;
4     uint8_t blue;
5     uint8_t alpha;
6 } Color;

```

---

Au début du programme, nous stockons cette structure sur la pile (stack). Nous l'initialisons ensuite avec les bons pins tels que décrits plus haut. Cette fonction modifie le mode des pins en `OUTPUT` après avoir initialisé la bibliothèque `WiringPi` que nous utilisons pour ce projet en raison de sa simplicité d'utilisation comparée à la bibliothèque `libgpiod` qui était proposée dans l'énoncé.

---

```

1 #define POWER_PIN 18
2 #define RED_PIN 12
3 #define GREEN_PIN 19
4 #define BLUE_PIN 13
5
6 int main(int argc, char *argv[]) {
7     Led led;
8
9     ...
10
11     if(init_led(&led, RED_PIN, GREEN_PIN, BLUE_PIN, POWER_PIN) != 0) {

```

---

```
12     fprintf(stderr, "Erreur lors de l'initialisation de la LED\n");
13     exit(1);
14 }
15 }
```

---

Nous disposons maintenant de la structure `Led`, sur laquelle nous pouvons effectuer des opérations telles que changer la couleur et l'éteindre (bien que nous n'utilisions pas cette dernière fonctionnalité).

## 2.3 Architecture et fonctionnement du programme

La logique de notre programme est séparée en plusieurs fichiers situés dans le dossier `src/` :

- `main.c` : contient le point d'entrée du programme.
- `math_suite.c` : implémente le cœur de l'algorithme de la suite de Conway.
- `led.c` : implémente l'affichage de la LED.
- `m_string.c` : implémente les fonctions relatives aux chaînes de caractères.
- `m_list.c` : implémente les fonctions relatives à la liste chaînée.

Chaque fichier, à l'exception de `main.c`, possède son fichier correspondant `.h` dans le dossier `include/` déclarant les structures et les signatures des fonctions relatives au fichier `.c`.

Faire clignoter la LED pendant l'exécution du programme nécessite une certaine coordination entre la fonction de résolution de la suite de Conway (`solve`) et la fonction permettant de changer la couleur de la LED. Le fonctionnement de la fonction `solve` peut être résumé à trois boucles imbriquées ; la solution naïve consiste à faire clignoter la LED après un certain nombre d'itérations. Cependant, cela ne garantit pas que la LED clignotera à intervalles réguliers et cela aura un impact sur les performances du programme. En effet, à chaque itération le programme devra incrémenter le compteur d'itérations, vérifier s'il doit faire clignoter la LED, et si c'est le cas, effectuer quatre appels système pour changer le statut des pins GPIO (un par canal).

Nous avons donc cherché à éviter ce coût au sein de la fonction `solve`. Pour cela, nous avons envisagé une solution avec deux flux d'exécution (threads) : un permettant de contrôler la LED et un réalisant le calcul de la fonction `solve`.

Nous avons également souhaité comparer l'implémentation avec la bibliothèque `pthread` et l'utilisation de `fork(2)`. C'est pourquoi notre programme peut être compilé avec plusieurs drapeaux, permettant de changer directement le code qui va être compilé à l'aide de macros. Les modes sont les suivants :

- `make all` : fait clignoter la LED au sein de la fonction `"solve"` (implémentation naïve).
- `make all THREAD=1` : utilise un thread de la bibliothèque `pthread` pour exécuter la fonction `solve`, le processus principal contrôle la LED.
- `make all FORK=1` : utilise `fork(2)`, le processus parent contrôle la LED tandis que le processus fils exécute la fonction `solve`.

### 2.3.1 Point d'entrée (`main.c`)

Ce fichier contient le point d'entrée du programme (fonction `main`). Celle-ci gère les différents paramètres passés en argument au programme, ouvre les descripteurs de fichiers si nécessaire, gère l'initialisation de la LED, exécute la fonction `solve` permettant de réaliser le calcul de la suite de Conway, gère les erreurs et ferme les descripteurs de fichiers proprement.

En premier lieu, le point d'entrée initialise la LED via la fonction `init_led`, configurant les 4 pins GPIO sur lesquels celle-ci est branchée. Les 4 pins sont des paramètres fixés du programme (constantes préprocesseur) ; il n'est donc pas possible de les changer sans recompiler le code.

Ensuite, le programme analyse les paramètres passés en arguments. Afin de pouvoir échouer rapidement si la syntaxe utilisée n'est pas correcte, il stocke dans quatre variables les arguments récupérés avant

d'ouvrir les fichiers.

## Analyse des arguments passés au programme

Nous utilisons une boucle `for` permettant d'itérer sur tous les arguments passés en paramètres et la fonction `strcmp(3)` afin de vérifier si le paramètre actuellement traité correspond à une option connue du programme. Si c'est le cas, la valeur de l'argument est validée (ex : vérification qu'un entier suit l'option `-debug`) et stockée dans une variable locale. Si une option est mal utilisée (argument manquant ou invalide), un message d'erreur est affiché sur la sortie d'erreur standard (`stderr`)<sup>2</sup> et le programme s'arrête avec le code d'erreur 1. Nous utilisons l'instruction `goto` pour sauter à la section gérant les erreurs au sein de la fonction `main`.

Une fois les arguments analysés, le programme ouvre les fichiers d'entrée (`input`) et de sortie (`output`) s'ils ont été passés en paramètres ; dans le cas contraire, le programme utilise `stdin` et `stdout` (entrée et sortie standard) comme descripteurs de fichiers.

Si l'ouverture d'un fichier retourne une erreur (fichier inexistant, permissions insuffisantes, etc.), le programme affiche un message explicatif sur la sortie d'erreur à l'aide de `perror(3)`. Nous utilisons ensuite l'instruction `goto` pour sauter à la section de gestion des erreurs.

## Gestion centralisée des ressources (`goto`)

Nous déclarons deux labels au sein de la fonction `main` : `cleanup` et `fail`.

Le premier (`cleanup`) contient le code de "nettoyage" de la fonction `main` ; c'est à cet endroit que nous fermons les descripteurs de fichiers. Cela nous permet de nous assurer que le programme ne contient aucune fuite de mémoire. Il quitte ensuite le programme avec le code d'erreur stocké dans la variable `ret`. Utiliser une variable pour stocker le code de retour permet de définir celui-ci sur une valeur différente de 0 en cas d'erreur d'exécution, tout en réutilisant la même logique de fermeture des ressources. Il s'agit d'ailleurs de la raison de l'utilisation de labels et de `goto` : cela permet d'éviter la duplication de code. Bien que `goto` soit parfois considéré comme une mauvaise pratique, son utilisation pour centraliser le nettoyage des ressources en C est idiomatique (le noyau Linux l'utilise abondamment).

Le second label (`fail`) contient la gestion des erreurs. Il est situé physiquement en dessous du 'return' du bloc `cleanup`, le code n'est donc jamais exécuté si `fail` n'est pas explicitement appelé. Le label `fail` met le code de retour `ret` à 1 (indiquant une erreur), change la couleur de la LED en rouge, puis réalise un saut (`goto`) vers le label `cleanup`. Cela permet de gérer les erreurs depuis n'importe quel endroit de la fonction : si une erreur survient, il suffit de sauter vers `fail`.

De plus, la fonction `cleanup` vérifie que les fichiers ont bien été ouverts avant de tenter de les fermer. Par exemple, si le fichier d'entrée est ouvert avec succès mais qu'une erreur survient lors de l'ouverture du fichier de sortie, la fonction sautera vers `fail` puis `cleanup`. Ce dernier fermera le fichier d'entrée (car ouvert), mais ignorera le fichier de sortie (car le pointeur est resté nul ou invalide).

### 2.3.2 Contrôle de la LED en utilisant `fork(2)`

Lorsque le programme est compilé avec l'argument `FORK=1` (`make all FORK=1`), `make(1)` compile en définissant le drapeau `USE_FORK` (via `-DUSE_FORK`). Le code contient ensuite des directives de préprocesseur (`#if USE_FORK ...`) qui permettent de compiler des lignes de code de manière conditionnelle.

Le code permettant de réaliser le `fork` est le suivant<sup>3</sup> :

---

2. Il est explicitement demandé dans les consignes du projet de ne pas écrire sur la sortie standard si le paramètre `-debug` est égal à 0 ou n'est pas communiqué. Cependant, puisque nous sommes au stade de l'analyse des arguments, nous ne connaissons pas encore la valeur de `debug`. De plus, les messages d'erreur sont écrits sur la sortie d'erreur (`stderr`) et non la sortie standard, ce qui est la bonne pratique pour signaler un mauvais usage.

3. La déclaration des variables a parfois été omise afin de simplifier la présentation du code. Le code source complet et fonctionnel se situe en annexe.

---

```

1  if(sigsetjmp(env, 1) == 0) {
2      signal(SIGCHLD, sig_handler);
3
4      pid = fork();
5
6      if (pid == -1) {
7          fprintf(stderr, "Erreur: impossible de fork()\n");
8          goto fail;
9      }
10
11     // Dans le child
12     if(pid == 0) {
13         if(debug >= 3) printf("Child pid=%d, ppid=%d\n", getpid(), getppid());
14         ret = solve(in, out, only_longest, debug);
15
16         goto cleanup;
17     } else {
18         if(debug >= 3) printf("Parent pid=%d\n", getpid());
19
20         ts.tv_sec = 0;
21         ts.tv_nsec = 500 * 1000 * 1000; /* 500ms */
22
23         while(1) {
24             set_color(&led, BLUE);
25             nanosleep(&ts, NULL);
26             set_color(&led, WHITE);
27             nanosleep(&ts, NULL);
28         }
29     }
30 }

```

---

Nous stockons d'abord l'environnement d'exécution via la fonction `sigsetjmp(3)`; celle-ci permet de réaliser un saut non local (équivalent d'un `goto` entre fonctions) directement depuis un gestionnaire de signal. En effet, nous devons utiliser `sigsetjmp(3)` et non `setjmp(3)` car la fonction `sig_handler` doit nécessairement se terminer pour que le masque de signaux soit restauré correctement par le système d'exploitation. Conformément à l'explication donnée dans le syllabus<sup>4</sup>, nous avons utilisé `sigsetjmp`.

La fonction `sig_handler` réalise le saut vers la fonction `main`.

---

```

1  sigjmp_buf env;
2
3  void sig_handler() {
4      siglongjmp(env, 1);
5  }

```

---

Lors de l'exécution initiale dans la fonction `main`, `sigsetjmp` stocke l'état actuel (registres, pile, masque de signaux) et retourne 0. Cela signifie que le bloc de la première condition réalisant le `fork` est alors exécuté. Tout d'abord, nous enregistrons notre gestionnaire de signal `sig_handler` à l'aide de `signal(2)`. Ce dernier écoute les signaux `SIGCHLD`, indiquant la mort d'un processus fils (créé avec `fork(2)`).

Ensuite, nous effectuons l'appel à `fork(2)`. Celui-ci réalise une duplication de processus. Le nouveau processus reprend l'exécution exactement à cet endroit. La valeur retournée (`pid`) dans le processus ayant initié le `fork` est l'identifiant de processus (`pid`) du fils nouvellement créé. Le fils, quant à lui, reçoit une valeur de retour de 0. Cela nous permet de déterminer qui doit exécuter quoi par la suite. Le fils exécute la fonction `solve`, avec les paramètres dupliqués du parent, tandis que le parent fait alterner la couleur de la LED entre bleu et blanc toutes les 500 millisecondes.

---

4. <https://icysi002.ig.umons.ac.be/Theorie/Fichiers/fichiers-signaux-extended.html>

Lors de la réception du signal par `sig_handler`, nous réalisons un `siglongjmp(3)` ; ce dernier permet de sauter directement là où le `sigsetjmp` a été appelé. La valeur de retour de la fonction est alors différente de 0 (ici 1), ce qui rend la condition `if` fausse. Le programme sort donc du bloc, ce qui interrompt immédiatement la boucle `while(1)`. De plus, la réception d'un signal interrompt les appels système bloquants tels que `nanosleep`, assurant une réactivité immédiate.

Lors de la compilation avec le drapeau `-Wall` de `gcc`, nous avons obtenu les avertissements suivants :

---

```

1  src/main.c:81:14: warning: variable 'in' might be clobbered by 'longjmp' or 'vfork' [-Wclobbered]
2      81 |         FILE* in = 0;
3  src/main.c:96:14: warning: variable 'ret' might be clobbered by 'longjmp' or 'vfork' [-Wclobbered]
4      96 |         int    ret = 0;
5  ...

```

---

Ces avertissements "Clobbered" (variable écrasée) sont dus au fait que l'appel à la fonction `sigsetjmp` sauvegarde les registres CPU. Le compilateur peut décider, pour des raisons d'optimisation, de garder le contenu des variables uniquement dans un registre, sans passer par la mémoire (RAM). Si la variable `in` se trouve dans un registre au moment où l'état est sauvegardé, puis est modifiée par la suite, sa valeur sera restaurée à l'état initial (ancien) lors du `siglongjmp`, écrasant ainsi la modification. Cela peut poser problème dans certains cas où la persistance de la modification est nécessaire. Cependant, dans notre architecture, nous ne modifions pas ces variables critiques entre le point de sauvegarde et le point de restauration dans le flux du parent.

Afin de supprimer l'avertissement de `gcc`, nous avons déclaré toutes les variables susceptibles d'être modifiées en mode `static`. Cela signifie qu'elles ne sont plus stockées dans les registres ou la pile, mais dans le segment de données du programme. Elles ne risquent donc plus d'être écrasées lors de la restauration des registres, car elles ne s'y trouvent plus.

Une fois l'exécution du fils terminée, nous utilisons `waitpid(2)` dans le parent, permettant de récupérer le code de sortie du fils à l'intérieur de la variable `status`. Étant donné que le processus fils retourne directement la sortie de la fonction `solve`, nous pouvons savoir si un problème a eu lieu lors de l'exécution de `solve`.

---

```

1  if(waitpid(pid, &status, 0) == -1) {
2      perror("wait(): erreur\n");
3      goto fail;
4  } else {
5      if (debug >= 3) printf("child a quitté avec %d\n", status);
6
7      if (status != 0) {
8          goto fail;
9      }
10 }

```

---

### 2.3.3 Contrôle de la LED en utilisant `pthread(3)`

Lorsque le programme est compilé avec l'argument `THREAD=1` (`make all THREAD=1`), et comme expliqué précédemment, `make(1)` définit le drapeau `USE_THREAD` lors de la compilation du programme. Les directives du préprocesseur se chargent ensuite d'inclure les lignes de code relatives à l'utilisation des threads.

L'exécution du thread est réalisée comme suit :

---

```

1  if(debug >= 3) printf("Execution du thread\n");
2  thread_args.in = in;
3  thread_args.out = out;

```

---

---

```

4  thread_args.only_longest = only_longest;
5  thread_args.debug = debug;
6  running = 1;
7  pthread_create(&thread, NULL, thread_solve, (void*)&thread_args);
8
9  while(running) {
10     set_color(&led, BLUE);
11
12     clock_gettime(CLOCK_REALTIME, &ts);
13     ts.tv_nsec += 500 * 1000 * 1000;
14
15     if (ts.tv_nsec >= 1000000000) {
16         ts.tv_nsec -= 1000000000;
17         ts.tv_sec += 1;
18     }
19
20     pthread_mutex_lock(&running_lock);
21     if (pthread_cond_timedwait(&running_cond, &running_lock, &ts) != ETIMEDOUT && !running) {
22         pthread_mutex_unlock(&running_lock);
23         break;
24     }
25
26     set_color(&led, WHITE);
27
28     clock_gettime(CLOCK_REALTIME, &ts);
29     ts.tv_nsec += 500 * 1000 * 1000;
30
31     if (ts.tv_nsec >= 1000000000) {
32         ts.tv_nsec -= 1000000000;
33         ts.tv_sec += 1;
34     }
35
36     if (pthread_cond_timedwait(&running_cond, &running_lock, &ts) != ETIMEDOUT && !running) {
37         pthread_mutex_unlock(&running_lock);
38         break;
39     }
40     pthread_mutex_unlock(&running_lock);
41 }
42

```

---

Nous avons besoin, pour gérer correctement le thread, de plusieurs variables globales supplémentaires. Le `mutex` (verrou) permet au programme de protéger l'accès concurrent à la variable partagée `running` et à la variable de condition `cond`. Ces variables sont définies de manière statique.

---

```

1  static pthread_mutex_t running_lock = PTHREAD_MUTEX_INITIALIZER;
2  static pthread_cond_t running_cond = PTHREAD_COND_INITIALIZER;
3
4  volatile int running;

```

---

La signature standard d'une fonction lancée par `pthread_create` n'accepte qu'un seul argument (un pointeur générique `void*`). Pour pallier cette limitation et transmettre plusieurs paramètres, nous créons une structure `t_args` que nousinstancions pour regrouper nos arguments, et nous transmettons son adresse au thread.

---

```

1  struct t_args {
2      FILE* in;
3      FILE* out;
4      int    only_longest;
5      int    debug;
6  };

```

---

Ladite fonction récupère les données de cette structure en convertissant le pointeur `void*` reçu vers le type `struct t_args*` pour pouvoir exécuter la fonction `solve`. Il est à noter que la valeur de retour de `solve` est sauvegardée et retournée à la fin du thread. Avant de terminer, le thread met à jour la variable `running` à 0 et signale ce changement via la variable de condition, ce qui permet au processus principal d'arrêter de faire clignoter la LED immédiatement.

---

```
1 void* thread_solve(void* args_void) {
2     struct t_args* args = (struct t_args*)args_void;
3
4     int ret = solve(args->in, args->out, args->only_longest, args->debug);
5
6     pthread_mutex_lock(&running_lock);
7     running = 0;
8     pthread_cond_signal(&running_cond);
9     pthread_mutex_unlock(&running_lock);
10
11     return (void*) ret;
12 }
```

---

Après l'arrêt de la boucle de la LED, le programme attend que le thread se termine proprement avec `pthread_join` et récupère sa valeur de retour.

---

```
1 void *thread_ret;
2 if (pthread_join(thread, &thread_ret) != 0) {
3     goto fail;
4 } else {
5     ret = (int) (long)thread_ret;
6     if (ret != 0) {
7         goto fail;
8     }
9 }
```

---

Une note importante concerne l'utilisation de la variable de condition dans la boucle gérant la LED. En effet, l'utilisation d'un `sleep` classique introduirait une latence : si le thread termine son exécution pendant que le processus principal dort, ce dernier ne s'en rendra compte qu'à la fin du délai (jusqu'à 500ms). Nous utilisons donc `pthread_cond_timedwait`, qui permet d'attendre un temps donné, mais qui peut être interrompu prématurément si le thread signale sa terminaison via `pthread_cond_signal`. Cela assure une réactivité immédiate à la fin du calcul.

Cette fonction prend également un temps absolu (horloge système) et non un temps relatif (durée). Nous récupérons donc l'instant actuel avant l'attente et nous lui ajoutons 500ms en veillant à gérer les débordements de nanosecondes car `tv_nsec` ne doit pas dépasser 999 999 999.

Finalement, il est important de noter que nous acquérons le mutex (*lock*) avant d'entrer dans l'attente de `pthread_cond_timedwait`. À première vue, cela pourrait sembler problématique : le mutex semblant acquis pour toute la durée de l'attente, le thread secondaire (celui exécutant `solve`) cherchant à notifier sa fin d'exécution se retrouverait bloqué en tentant d'acquérir ce même mutex, générant une interblocage (*deadlock*) tant que le thread principal ne l'aurait pas libéré.

Cependant, le fonctionnement interne de `pthread_cond_timedwait` résout ce problème : la fonction relâche atomiquement le mutex au moment où elle met le thread en sommeil. Une fois le signal reçu (ou le temps écoulé), la fonction ré-acquiert automatiquement le mutex avant de rendre la main au programme. Ce mécanisme garantit deux choses : premièrement, que le thread secondaire peut acquérir le mutex pour signaler sa fin sans latence pendant que le principal dort ; deuxièmement, que l'accès à la variable partagée `running` reste parfaitement synchronisé et protégé contre les accès concurrents ("race conditions") au réveil des threads.

### 2.3.4 Contrôle de la LED dans la fonction solve

Sans l'utilisation de `fork(2)` ou de `thread`, la mise à jour de la LED se fait directement dans la fonction `solve`.

Afin de vérifier que la LED doit être gérée dans ladite fonction, le fichier d'en-tête `math_suite.h` inclut la directive de préprocesseur `#if/#endif`.

---

```

1  #if !defined(USE_FORK) && !defined(USE_THREAD) && !defined(NO_LED)
2      #define LED_SOLVE
3  #endif

```

---

Le code ci-dessus définit la macro `LED_SOLVE`<sup>5</sup> lorsqu'aucun des drapeaux évoqués dans les sections précédentes n'est ajouté à la compilation par le `Makefile`, qui gère ici également le potentiel ajout d'un drapeau `NO_LED`.

Ensuite, la fonction `solve` est définie comme pouvant prendre quatre ou cinq arguments, selon que la définition de `LED_SOLVE` a eu lieu ou non, et ce en utilisant la directive de préprocesseur `#ifdef/#endif`.

---

```

1  int solve(FILE* in,
2          FILE* out,
3          int    only_longest,
4          int    debug
5          #ifdef LED_SOLVE
6          ,Led* led
7          #endif
8          );

```

---

Le corps de la fonction `solve` contient enfin également la directive `#ifdef/#endif` afin de n'effectuer des actions sur la LED que lorsque la macro `LED_SOLVE` est définie.

À l'appel de la fonction avec le paramètre `led` présent, une variable `unsigned long long led_iter` est définie pour compter chaque itération de la fonction. Après un certain nombre d'itérations, la couleur de la LED est alors mise à jour en alternance.

---

```

1  #ifdef LED_SOLVE
2      led_iter++;
3      if (led_iter % 10000000 == 0) {
4          set_color(led, BLUE);
5      } else if (led_iter % 10000000 == 5000000) {
6          set_color(led, WHITE);
7      }
8  #endif

```

---

Le nombre choisi de 10 000 000 itérations est arbitraire et basé sur des tests permettant simplement de bien discerner le clignotement de la LED lors de l'exécution du programme.

Contrairement aux méthodes précédentes, celle-ci ne fait pas clignoter la LED toutes les 500ms, ce qui peut mener à des différences de périodes de clignotement en fonction de la charge processeur et de la vitesse de calcul.

### 2.3.5 Calcul de la suite de Conway (math\_suite.c)

Le fichier `math_suite.c` contient la déclaration de la fonction `solve(FILE* in, FILE* out, int only_longest, int debug)`. Celle-ci prend en paramètre le fichier d'entrée "in", le fichier de sortie

---

5. Ce nom venant purement du fait de gérer la LED dans la fonction `solve`.

"out", ainsi que deux paramètres supplémentaires : l'indication de n'afficher que les résultats les plus longs et le niveau de débogage à utiliser.

La fonction réalise l'allocation de deux chaînes de caractères `m_string : s` et `s_new`. Ces structures sont stockées sur la pile d'exécution, bien que le contenu de chaque chaîne de caractères soit alloué dynamiquement à l'aide de `calloc(3)`. Nous effectuons ces allocations une seule fois au début du traitement, car nous avons évalué que seules deux chaînes de caractères étaient nécessaires pour la réalisation de l'algorithme : la première contenant l'itéré  $i$  et la seconde l'itéré  $i+1$  en train d'être généré. Nous ne pouvons malheureusement pas utiliser une seule chaîne de caractères car l'itéré  $i+1$  possède généralement une longueur plus grande que l'itéré  $i$ . Cela signifie que si nous modifions directement la chaîne `s`, nous écraserions une partie des données contenues dans l'entrée avant même d'avoir pu les traiter.

## Lecture de l'entrée

Nous avons également implémenté une fonction `read_input(FILE* in, m_string* s, int* iter)` permettant de lire une entrée depuis le flux pointé par `"in"` et de stocker l'itéré initial dans la chaîne `"s"` ainsi que le nombre d'itérations à réaliser dans `"iter"`. Cette fonction retourne 0 si la lecture s'est déroulée sans encombre ou `-1` si la fin du fichier (EOF) ou une erreur a été rencontrée.

Nous réalisons une lecture caractère par caractère, via `fgetc(3)`, ce qui permet de détecter les espaces (séparateur entre l'itéré initial et le nombre d'itérations à réaliser) ainsi que le séparateur de ligne `n`, indiquant la fin d'une entrée. Nous copions ensuite, caractère par caractère, l'entrée dans la chaîne `"s"`. Si la taille allouée de la chaîne de caractères n'est pas suffisante, la fonction augmentera automatiquement la capacité de la chaîne via une réallocation. Elle pourra également retourner `-1` si la réallocation ne s'est pas déroulée correctement (en raison d'un manque de mémoire par exemple).

---

```

1 zero_string(s);
2 while(1) {
3     c = fgetc(in);
4     if (c == EOF) return -1;
5     if (c == ' ') break;
6     if(string_check(s, i) == -1) return -1;
7
8     s->ptr[i++] = c;
9 }

```

---

La fonction `solve` utilise donc `read_input` pour lire l'itéré à traiter. Tant que la fonction fournit un itéré valide, la boucle de résolution continue.

## Algorithme

Nous nous sommes basés sur le code fourni dans l'exemple Python pour concevoir notre algorithme. Celui-ci fonctionne de manière similaire à la version Python mais utilise un mécanisme différent pour comptabiliser le nombre de chiffres répétés consécutivement.

---

```

1 while(read_input(in, &s, &iter) == 0) {
2     for(i = 0; i < iter; i++) {
3         j = 0; end = 0;
4         while(j < s.max_size && s.ptr[j] != '\0') {
5             start = end;
6             /* Comptage du nombre de chiffres identiques */
7             if(increment(&s_new, start, &end) == -1) {
8                 ret = -1; goto clean;
9             }

```

---

```

10     while (++j < s.max_size && s.ptr[j] != '\0' && s.ptr[j-1] == s.ptr[j]) {
11         if(increment(&s_new, start, &end) == -1) {
12             ret = -1; goto clean;
13         }
14     }
15     /* Réallocation de la chaîne de caractères si espace insuffisant */
16     if (string_check(&s_new, end+1) == -1) {
17         ret = -1; goto clean;
18     }
19     /* Ajout du chiffre compté à la fin */
20     s_new.ptr[++end] = s.ptr[j-1];
21     end++;
22 }
23
24 swap_string(&s, &s_new);
25 zero_string(&s_new);
26 }
27
28 fflush(out);
29 print_string(out, &s, end);
30 }

```

La version Python utilise un compteur entier "j" et l'ajoute ensuite à la chaîne de caractères via `+= str(...)`. En C, la conversion d'entier en chaîne de caractères et la concaténation ne sont pas aussi triviales à réaliser qu'en Python, notamment lorsque les chaînes résultantes sont de longueur variable. Il est en effet possible que le nombre de répétitions d'un chiffre soit supérieur ou égal à 10, ce qui nécessite plusieurs caractères pour représenter le compte.

Pour gérer ce cas, nous avons décidé de travailler directement sur les chaînes de caractères en utilisant deux indices : `start` et `end`, ainsi qu'une fonction `increment(m_string *s, size_t start, size_t *end)`. Les indices permettent de délimiter une sous-chaîne de `s_new`, qui représente le nombre d'occurrences comptées en représentation ASCII.

Plutôt que de calculer le nombre d'occurrences dans une variable entière `int` puis de la convertir (via `sprintf`), nous écrivons la valeur "1" dans la chaîne, et nous appelons `increment` à chaque fois qu'un caractère identique est rencontré. Cette fonction agit comme une unité arithmétique et logique (ALU) simplifiée travaillant directement sur les codes ASCII :

- Elle parcourt la sous-chaîne de la droite vers la gauche (de `end` vers `start`).
- Si le caractère est inférieur à '9', elle l'incrmente simplement.
- Si le caractère est '9', elle gère la retenue en mettant le caractère à '0' et en passant au caractère précédent.
- Dans le cas critique où une retenue se propage jusqu'au début du nombre (exemple : passage de "9" à "10" ou "99" à "100"), la fonction effectue un décalage mémoire de tout le reste de la chaîne vers la droite pour insérer le nouveau chiffre '1'.

Cette approche, bien que plus complexe à implémenter qu'un simple compteur, nous affranchit totalement des limites des types entiers du processeur (`uint64_t`). Nous pouvons théoriquement compter un nombre d'occurrences ayant des millions de chiffres.

Une fois que l'itéré  $i+1$  a été calculé, l'implémentation Python copie le contenu de la nouvelle chaîne de caractères dans la chaîne source, permettant de réaliser de manière itérative le calcul suivant. Au lieu de recopier le contenu entier de chaque chaîne de caractères (ce qui demande une copie caractère par caractère coûteuse), nous réalisons simplement un échange ("swap") des pointeurs de chaque structure `m_string`. Cela permet de réaliser une "copie" à coût constant (seules les adresses mémoire et les tailles sont échangées).

Avant d'afficher le résultat via la fonction `print_string`, nous devons vider (`fflush(3)`) le buffer interne de la bibliothèque `libc` comme expliqué dans la section correspondante (section 2.1.1).

Concernant l'utilisation des instructions de sauts ("goto"), celles-ci permettent de simplifier la sortie de la fonction en s'assurant que les chaînes de caractères allouées à l'aide de `calloc(3)` sont bien libérées via `free(3)` avant que la fonction ne retourne. En effet, si nous avons simplement utilisé `return -1` de manière naïve, les chaînes de caractères allouées dans la fonction n'auraient jamais été libérées, ce qui aurait généré une fuite de mémoire.

Pour cela, nous avons labellisé le bloc de fin de la fonction "`clean`". Celui-ci réalise la désallocation mémoire des deux chaînes de caractères avant de quitter la fonction, ce qui garantit une gestion correcte des ressources. Si la fonction n'appelle jamais le `goto`, le code de retour `ret` est mis à 0 et le code s'exécute de la même manière, traversant le label sans encombre. Cela permet une gestion plus simple et centralisée que la duplication des appels à `free` à chaque endroit où la fonction est susceptible de retourner une erreur.

---

```

1     ret = 0;
2     clean:
3         free_string(&s);
4         free_string(&s_new);
5
6     return ret;

```

---

## Affichage des résultats les plus longs uniquement

Pour l'affichage des résultats les plus longs uniquement, nous avons utilisé la liste chaînée `m_list` (section 2.1.3). Nous y stockons les "meilleurs" éléments trouvés jusqu'à présent. Lorsqu'un élément supérieur à ceux stockés est trouvé, nous vidons la liste et ajoutons le nouvel élément. Une fois que toutes les entrées ont été traitées, la liste ne contient plus que les éléments gagnants.

Nous avons mis en place une fonction permettant de comparer deux résultats : `int compare_result(compare_key k1, compare_key k2)`. Celle-ci vérifie d'abord le nombre de chiffres différents (taille), et ensuite le nombre de caractères différents (taille) pour départager deux résultats. Sa valeur de retour est 0 si `k1 == k2`, 1 si `k1 > k2` et -1 si `k1 < k2`.

Au lieu de comparer directement des chaînes de caractères (ce qui serait coûteux), nous comparons des "clés de comparaison" (`compare_key`). Celles-ci contiennent les métadonnées nécessaires à la comparaison : la taille de la chaîne de caractères (`length`) et le nombre de chiffres différents (`ndiff`).

---

```

1     typedef struct compare_key {
2         size_t length;
3         int ndiff;
4     } compare_key;

```

---

Ces clés de comparaison sont obtenues via la fonction `get_compare_key`.

---

```

1     void get_compare_key(compare_key* k, m_string s) {
2         short n;
3         size_t i;
4
5         n = 0;
6         for (i = 0; s.ptr[i] != '\0'; i++) {
7             n |= (1 << (s.ptr[i] - '0'));
8         }
9
10        k->length = i;
11        k->ndiff = 0;
12    }

```

---

---

```

13     for (i = 0; i < 10; i++) {
14         if ((n>>i) & 1) == 1) {
15             k->ndiff++;
16         }
17     }
18 }

```

---

Dans cette fonction, nous utilisons un bitmask `n` afin de compter le nombre de chiffres différents. Le bit correspondant au chiffre `i` présent dans la chaîne est mis à 1. Nous utilisons l'expression `s.ptr[i] - '0'` pour obtenir la valeur entière du chiffre (distance ASCII entre le caractère et '0') et l'utiliser comme indice de décalage binaire. Ensuite, nous comptabilisons le nombre de bits à 1 dans le masque pour déterminer le nombre de chiffres uniques.

Comparer des clés au lieu de chaînes de caractères nous permet de ne stocker que la clé de comparaison "la plus grande" trouvée jusqu'à présent. À chaque fois qu'une nouvelle entrée est calculée, nous comparons son résultat à cette clé de référence. S'il est supérieur, nous vidons la liste, ajoutons le nouvel élément et mettons à jour la clé de référence. S'il est égal, nous l'ajoutons simplement à la liste existante. S'il est inférieur, nous l'ignorons.

Étant donné que la chaîne de caractères actuellement stockée dans la liste ne peut pas être réutilisée pour le calcul suivant (car elle appartient désormais à la liste), nous devons allouer une nouvelle chaîne de travail. Nous évitons les fuites de mémoire en nous assurant que la fonction `clear_list` désalloue correctement chaque chaîne contenue dans la liste lors de la suppression.

---

```

1  switch (compare(&longest_key, s)) {
2      case 1:
3          clear_list(&longest);
4      case 0:
5          push_back(&longest, s);
6
7          if (alloc_string(&s) == -1) {
8              ret = -1; goto clean;
9          }
10 }

```

---

## 2.4 Compilation

Nous utilisons `make(1)` pour compiler l'exécutable final du programme. Celui-ci compile de manière indépendante chaque fichier ".c" situés dans `src/` en fichier objet ".o" dans `obj/`. Nous "linkons" ensuite tous les fichiers objets ensemble pour créer un exécutable linux (ELF) à l'emplacement `bin/math_suite`. Lors de la compilation, nous utilisons l'option `-I` de `gcc(1)` permettant d'indiquer au compilateur que nous avons déclaré des fichiers d'entête supplémentaires dans `include/`.

### 2.4.1 Instructions de compilation

Pour compiler le programme, il faut tout d'abord s'assurer de posséder les prérequis :

---

```

1  sudo apt update && sudo apt install build-essential

```

---

Cette commande permet d'installer les différents outils (`make`, `gcc`, ...) nécessaires à la compilation du programme.

Ensuite, il faut installer manuellement la librairie WiringPi permettant la manipulation des pins GPIO, c'est à dire de la LED.

---

```

1  wget https://github.com/WiringPi/WiringPi/releases/download/3.16/wiringpi_3.16_arm64.deb

```

---

---

```
2 sudo apt install ./wiringpi_3.16_arm64.deb
```

---

Pour compiler le programme en utilisant le mode par défaut de contrôle de la LED dans la fonction solve, il faut utiliser la commande :

---

```
1 make all
```

---

L'exécutable `bin/math_suite` est alors généré<sup>6</sup>.

Pour compiler avec le mode thread ou fork de gestion de la led :

---

```
1 make all THREAD=1
2 make all FORK=1
```

---

Par défaut, le compilateur est configuré pour optimiser le programme au maximum (-O3). Il est possible de compiler avec les symboles de débogages en rajoutant le paramètre `DEBUG=1` à la fin de la commande `make`.

La commande `make clean` permet de supprimer tout ce qui a été généré par la compilation, c'est à dire le dossier `bin/` et le dossier `obj/`. Cette commande doit être exécutée avant de compiler à nouveau le programme, pour s'assurer qu'aucun fichier résiduel ne pose problème.

## 3 Validation et tests

Nous avons validé le comportement de notre programme en vérifiant que celui-ci se comportait comme attendu dans plusieurs cas différents.

### 3.1 Validation des arguments

Nous avons testés le programme avec des paramètres valides :

- Les arguments `-input` et `-output` avec des fichiers valides (existant et inexistant pour `-output`)
- L'argument `-input` seul
- L'argument `-output` seul avec des fichiers valides (existants et inexistant pour `-output`)
- Pas d'argument pour `input` pour vérifier la lecture de l'entrée standard
- L'argument `-debug`
- L'argument `-only-longest`
- Plusieurs combinaisons de tous les arguments, dans des ordres différents.

Nous avons également testé des paramètres invalides pour valider le comportement du programme :

- Un argument manquant pour `-input`, `-output` et `-debug`
- Des valeurs non-numériques pour `-debug`
- Un fichier inexistant ou avec de mauvaises permissions pour `-input` et `-output`, afin de vérifier l'affichage des erreurs.

Dans tous les cas d'erreur, le programme s'est comporté comme prévu : la LED est devenue rouge et le code d'erreur retourné était 1.

---

6. Pour exécuter ce programme, il est nécessaire que l'utilisateur soit membre du groupe `gpio`. Dans le cas contraire, le programme refusera de se lancer, car il n'aura pas accès au contrôle de la LED.

### 3.2 Validation des résultats

Nous avons validé les résultats de notre algorithme en le comparant à l'exemple Python.

Nous avons utilisé plusieurs combinaisons d'entrées différentes et les avons exécutées (avec et sans `-only-longest`) avec l'implémentation Python et avec notre programme. Afin de ne pas comparer des entrées (potentiellement longues) caractère par caractère, nous avons haché les sorties avec la commande `sha256sum(1)` et comparé directement les valeurs des hash. Pour deux sorties identiques, les hash sont identiques, cependant si les sorties sont différentes ne seraient-ce que de un caractère, alors les hash seront différents.

---

```

1 pi@group2:~/projet/projet $ echo "2031 33" > ./bin/math_suite | sha256sum
2 e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855 -
3 pi@group2:~/projet/projet $ echo "2031 33" > python3 math_suite.py | sha256sum
4 e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855 -

```

---

### 3.3 Vérification des fuites de mémoires

Pour s'assurer que le programme ne contenait aucune fuite de mémoire, nous avons utilisé `valgrind(1)`.

Nous avons lancé `valgrind` avec plusieurs combinaisons de paramètres différents (dont des paramètres générant explicitement des erreurs) pour nous assurer que le programme ne contenait pas de fuites de mémoires. Nous avons utilisé `-trace-children` afin de suivre le fork.

---

```

1 valgrind --trace-children=yes --child-silent-after-fork=no \
2 --leak-check=full --show-leak-kinds=all ./bin/math_suite --input input.txt

```

---

Dans tous les cas testés, `valgrind`, affiche correctement que tous les blocs de mémoire alloués ont été libérés, ce qui signifie qu'aucune fuite de mémoire n'est présente.

Tous les tests ont ensuite été réitérés pour les 3 modes de compilation du programme : `fork`, `thread` et `"solve"`.

## 4 Performance

Nous avons souhaité mesurer la performance de notre programme, afin de comparer les différentes implémentations de gestion de la LED et le script Python.

### 4.1 Méthodologie

Pour pouvoir mesurer la performance de notre programme, nous devons d'abord établir l'environnement dans lequel les tests seront effectués.

Nous avons réalisé les tests sur un Raspberry Pi 4 modèle B, version 4Go de RAM. Le Raspberry Pi était alimenté par le bloc d'alimentation officiel en USB-C et connecté au réseau local par l'intermédiaire d'un câble RJ45. Les 4 pins définis à la table 1 étaient branchés pour relier la LED. Une carte micro-SD de 64Go contenant le système d'exploitation était insérée dans le Raspberry Pi.

Avant d'effectuer tout test, nous avons redémarré le Raspberry Pi afin d'éviter d'avoir des services en exécution qui ne seraient pas nécessaires au bon fonctionnement de celui-ci. Afin d'éviter que le système ne soit encore en train de charger des services au démarrage, nous attendons systématiquement 10 minutes avant de commencer les tests.

Nous avons souhaité limiter le nombre de services actifs sur le système. Le seul service additionnel aux services système par défaut de la distribution installée (Debian 6.12.47 bookworm) est `Tailscale`, que nous utilisons pour pouvoir accéder au Raspberry Pi depuis l'extérieur du réseau local.

Une fois le système démarré et stabilisé (vérifié via l'utilitaire `htop` en observant l'activité du processeur), nous pouvons démarrer les tests. Ceux-ci sont lancés depuis une session SSH.

Afin d'éviter toute erreur humaine, nous avons créé un script `bash` automatisant l'exécution des tests. Ce script impose une attente de 10 minutes entre chaque test pour laisser le processeur refroidir (le Raspberry Pi n'étant pas muni d'une solution de refroidissement active, cela évite le *thermal throttling*).

Afin d'éviter toute erreur humaine, nous avons créé un script `bash` automatisant l'exécution des tests. Ce script impose une attente de 10 minutes entre chaque test pour laisser le processeur refroidir (le Raspberry Pi n'étant pas muni d'une solution de refroidissement active, cela évite le *thermal throttling*).

Pour chaque méthode présentée précédemment, nous recompilerons le programme avec le mode adéquat avant d'effectuer les mesures. Les tests effectués sont :

- Exécution normale avec `input.txt`
- Exécution avec le drapeau `-only-longest` avec `input.txt`
- Exécution normale avec `input_slow.txt`
- Exécution avec le drapeau `-only-longest` avec `input_slow.txt`

Les fichiers d'entrée sont :

```
# fichier input.txt ("normal")
1 5
1 10
25 3
2195 2
11113333 1 # chaque 1 et 3 est répété un grand nombre de fois
1 10

# fichier input_slow.txt ("slow")
1 66
```

Tous ces tests sont exécutés à l'aide du programme `perf` de telle sorte :

---

```
1 perf stat -r 10 -d ./bin/math_suite --input input.txt > /dev/null
```

---

Tous ces tests sont exécutés à l'aide de l'outil d'analyse de performance `perf` de la manière suivante :

---

```
1 perf stat -r 10 -d ./bin/math_suite --input input.txt > /dev/null
```

---

Cette commande exécute le programme 10 fois (`-r 10`) pour récolter des statistiques moyennées sur un échantillon, minimisant ainsi les risques de données aberrantes. Nous redirigeons la sortie standard du programme vers `/dev/null` pour éviter que l'affichage des chaînes de caractères dans le terminal n'impacte les performances mesurées (I/O bound).

## 4.2 Mesure selon l'implémentation

Nous obtenons, une fois les tests exécutés, une série de résultats avec un grand nombre d'information. Seule une partie de ces informations nous intéressent, le temps total prit par le programme et le nombre total d'instructions exécutées. Ces résultat sont affichées dans la table 2 pour chaque implémentation.

L'outil `perf(1)` nous permet de comparer de manière plus fiable les différents résultats en nous fournissant le nombre d'instructions exécutées par le CPU. En effet, si le temps d'exécution peut dépendre de nombreux facteurs externes (durée d'appel système, latence du stockage, préemption par d'autres

|        | normal  |              |  | normal only-longest |              |  | slow           |              |  | slow only-longest |              |  |
|--------|---------|--------------|--|---------------------|--------------|--|----------------|--------------|--|-------------------|--------------|--|
| fork   | 0,0035  | secondes     |  | 0,0045              | secondes     |  | 6,3196         | secondes     |  | 6,4806            | secondes     |  |
|        | 651 263 | instructions |  | 653 156             | instructions |  | 20 804 605 592 | instructions |  | 21 846 550 588    | instructions |  |
| thread | 0,0032  | secondes     |  | 0,0033              | secondes     |  | 6,3659         | secondes     |  | 6,5271            | secondes     |  |
|        | 869 809 | instructions |  | 871 706             | instructions |  | 20 804 618 872 | instructions |  | 21 846 564 374    | instructions |  |
| solve  | 0,0031  | secondes     |  | 0,0027              | secondes     |  | 6,9309         | secondes     |  | 7,0892            | secondes     |  |
|        | 642 130 | instructions |  | 644 014             | instructions |  | 23 417 048 368 | instructions |  | 24 458 993 250    | instructions |  |

**TABLE 2** – Résultats des tests pour chaque implémentation

processus, *thermal throttling*...), le nombre d'instructions reflète la complexité algorithmique réelle exécutée par le processeur, de manière indépendante du temps horloge. Par exemple, si le processus est mis en suspens par l'ordonnanceur pendant un long moment, ce compteur ne s'incrémente pas, contrairement au temps écoulé.

Nous constatons que, pour une entrée de taille normale, il n'y a pas de différences notables entre les implémentations. L'implémentation "solve" est plus rapide de 0,4 ms et utilise 3 000 instructions CPU de moins que le "fork" pour le même résultat. Cette différence anecdotique de 3 000 instructions s'explique par le coût fixe de la duplication du processus (`fork`). Nous constatons également que "thread" est similaire à "fork", tout en utilisant 178 000 instructions CPU de plus sur les petits fichiers. Nous attribuons cela au surcoût de la bibliothèque `pthread` et de la gestion des mutex.

L'impact du paramètre `-only-longest` est quant à lui très limité sur le nombre d'instructions CPU (+2 000) pour chaque implémentation, ce qui témoigne de l'efficacité de notre implémentation de la liste chaînée. Nous observons toutefois des variations plus importantes sur les temps d'exécution, ce qui est attendu étant donné la faible durée totale du test.

Nous constatons ensuite que, pour l'entrée "slow", l'écart entre l'implémentation "solve" et les deux autres se creuse de manière notable (+600ms, soit 9%), ce qui confirme bien l'impact négatif de la vérification du compteur d'itérations au sein de la boucle critique de calcul. Le fait de posséder un fil d'exécution dédié (thread ou processus fils) à la mise à jour de la LED permet de supprimer cette vérification dans la boucle de calcul et d'augmenter les performances. Plus le nombre d'itérations demandé sera important, plus l'écart sera visible.

Un écart de 40ms persiste entre les implémentations "thread" et "fork", "fork" étant légèrement plus rapide. A priori, nous pensions que "fork" serait plus lent que "thread" en raison du coût plus conséquent de la duplication de processus. Cependant, l'absence de nécessité de synchronisation continue dans l'implémentation de "fork" (utilisation de signaux et saut non local uniquement à la fin) donne un léger avantage à cette solution sur le long terme par rapport à la gestion des mutex des threads.

Nous en concluons que la meilleure implémentation dépend de l'entrée du programme. Pour une entrée triviale, la solution "solve" est la plus directe. Pour des calculs lourds générant une suite de Conway massive, les solutions "fork" et "thread" sont préférables, avec un léger avantage pour "fork".

### 4.3 Comparaison avec l'implémentation Python

Nous pouvons maintenant comparer nos implémentations avec celle de référence en Python. La table 3 montre les résultats obtenus sur cette implémentation.

|        | normal      |              |  | normal only-longest |              |  | slow            |              |  | slow only-longest |              |  |
|--------|-------------|--------------|--|---------------------|--------------|--|-----------------|--------------|--|-------------------|--------------|--|
| python | 0,1116      | secondes     |  | 0,1076              | secondes     |  | 155,82          | secondes     |  | 151,00            | secondes     |  |
|        | 101 518 075 | instructions |  | 101 651 108         | instructions |  | 576 834 871 545 | instructions |  | 577 204 407 883   | instructions |  |

**TABLE 3** – Résultats sur l'implémentation Python

Il est évident que notre implémentation C est bien plus performante que l'implémentation naïve en Python. En moyenne, notre implémentation est **25 fois plus rapide** que Python et utilise environ 27 fois moins d'instructions pour obtenir le même résultat, et ce, même avec la gestion supplémentaire de la LED (absente du script Python fourni).

## 5 Conclusion

A travers les différentes méthodes d'implémentation du calcul de la suite de Conway (méthode directe, méthode fork et méthode thread), ce projet démontre l'efficacité temporelle et spatiale du **C** par rapport à **python**.

Nous notons toutefois que le programme C est beaucoup plus complexe que le programme Python (environ 1000 lignes de code contre 60 pour Python). Le C demande au développeur de gérer lui-même la mémoire, optimisations, et fournit une bibliothèque standard très limitée comparée à celle du langage Python. Avoir un langage bas niveau rend la conception du programme plus complexe, mais permet un gain de performance très important. @a